Knowing How Logics

Rens Doodeman, Ziqi Wang, and Zixuan Chen

Monday 16th March, 2026

Abstract

TODO!

Contents

1 How to use this? **2**

2 Project Description **2**

3 Single Agent **3**

4 Multi Agent **6**

5 Tests for Semantics **7**

6 Conclusion **8**

Bibliography **8**

1 How to use this?

To generate the PDF, open `report.tex` in your favorite LaTeXeditor and compile. Alternatively, you can manually do `pdflatex report; bibtex report; pdflatex report; pdflatex report` in a terminal.

You should have stack installed (see `https://haskellstack.org/`) and open a terminal in the same folder.

- To compile everything: `stack build`.
- To open ghci and play with your code: `stack ghci`
- To run the executable from Section ??: `stack build && stack exec myprogram`
- To run the tests from Section ??: `stack clean && stack test --coverage`

2 Project Description

Ability-based epistemic logics are found many places in computer science, as arguably they provide us with a formal approach for strategic reasoning and AI planning. Several logics in this family have been studied lately. In particular, epistemic logics describing the notion of *knowing how* received a lot of attention, due to their simplicity and good meta-logical properties.

The simplest version of *knowing how*, $\mathcal{L}_{Kh}$, based on the paper by (Wang 2015[Wan15]), with the semantics reformulated in (Areces et al. 2021[AFSVQ21]). This logic captures the agent's ability to guarantee the completion of a task under a given condition. This idea is ideal in a sense, since even though the agent manages to do something under a condition, it does not imply the agent really know how. For example, one can win a lottery does not imply one can guarantee how to win. The uncertainty is involved in this case. Therefore, another logic may be prefered, $\mathcal{L}^U_{Kh}$. In this logic, an agent may have *indistinguishability* among several plans they might take, a critical concept in epistemic logic. Indistinguishable plans form equivalence classes, partitioning the set of available plans. Intuitively, plans considered identical by the agent are placed in the same class. Furthermore, this logic involves multi-agents, as opposed to a single agent. Finally, a new logic, $reg - \mathcal{L}^U_{Kh}$, a variation of the second version, introduces equivalence classes that are defined in terms of finite automatons.

In this project, we aim to model this family of knowing-how logics. We also intend to implement the model checking for the logic $reg - \mathcal{L}^U_{Kh}$ defined in [DF23]. Our project goals are defined as follows.

- Get familiar with the family of knowing-how logics;
- Implement the the family of knowing-how logics (syntax, semantics) in Haskell;
- Implement testing for knowing-how languages;
- (Optional) Implement model checking problem for the logic $reg - \mathcal{L}^U_{Kh}$ defined in [DF23];
- (Optional) Implement testing for model checking.

- (Optional) Implement a user-interface or at the very least an executable (using parsing) for model checking.

3 Single Agent

In this section we model the language of Knowing How L_{KH} as by the definition of Y. Wang [Wan15].

Given a set of proposition letters P, the language L_{KH} is defined as follows s.t. $Kh(\psi, \varphi)$ is the modality expressing "the agent knows how to achieve φ given ψ". The abbreviations $\bot$, $\vee$, $\rightarrow$, and the universal modality $U_\varphi := Kh(\neg\varphi, \bot)$ have been left out for simplicity.

```
module SingleAgent where

import Data.List (nub, delete)
-- import NoneEmpty including its constructor :|
import Data.List.NonEmpty (NonEmpty(..), toList)
import Test.QuickCheck

type Proposition = Int

data Form = P Proposition | Neg Form | Conj Form Form | KH Form Form | T
    deriving (Eq, Show, Ord)
```

Instead of a Kripke semantics, the Logic opts for a labelled transition system (LTS) $(\mathcal{S}, \mathcal{R}, \mathcal{V})$ s.t.

- $\mathcal{S}$ is a non-empty set of states
- $\mathcal{R} : \Sigma \rightarrow 2^{\mathcal{S}\times\mathcal{S}}$ is a collection of transitions labelled by actions in Σ
- $\mathcal{V} : \mathcal{S} \rightarrow 2^P$ is a valuation function

Additionally $s \xrightarrow{a} t$ if $(s, t) \in R(a)$, being the relation or 'transition' from s to t. This is further modelled after this code snippet. In the literature by Y. Wang the LTS is called an ability map.

When creating a LTS, consider that states :: NonEmpty State, and therefore must use the :| constructor, i.e. (n :| [n+1..]). For more information see NonEmpty on Hoogle.

```
type Action = Integer
type Plan = [Action]
type State = Int
type Valuation = [(State, [Proposition])]

data AbilityMap = LTS {
    states :: NonEmpty State,
    transitions :: Relations,
    valuation :: Valuation
} deriving(Show)
```

In (Wang2015), two relational notions are introduced:

- R_a, the atomic relation associated with an action a;
- R_σ, the composite relation associated with a plan σ.

Both of these are binary relations on states. Therefore, in our implementation, we use the same Haskell type `Rel` to represent them, since both can be understood as sets of pairs of states.

To represent the family of atomic relations $R = (R_a)_a$, we index relations by actions. This gives rise to the type `Relations`, corresponding to the collection of action-labelled transitions in the literature.

We do not model R_σ explicitly as a separate data structure. Instead, we only implement the operations needed to reason about plan execution and strong executability.

For this purpose, we define the following helper functions:

- `image`, which computes the image of a state under a relation, i.e. it collects all successor states of a given state;
- `r_a`, which takes an action a and the family of relations R, and returns the indexed relation R_a;
- `executePlan`, which computes all possible end states obtained after executing a plan from a given initial state;
- `stronglyExecutableAt`, which determines whether a plan is strongly executable at a given state.

```haskell
-- Both $R_a$ and $R_\sigma$ share the same type.
-- Here we model the whole set of relations, without classification.
type Rel = [(State, State)]

-- The family of action-indexed relations (R_a)_a
type Relations = [(Action, Rel)]

-- Image of a state under R_a
image :: Rel -> State -> [State]
image r u = [v | (x, v) <- r, x == u]

-- Given (R_a)_a and an action x, return R_x
r_a :: Relations -> Action -> Rel
r_a rs a =
    case lookup a rs of
        Just r  -> r
        Nothing -> []

-- Execute a plan from a given start state
-- This corresponds to R_sigma(u)
executePlan :: Relations -> State -> Plan -> [State]
executePlan _  u []       = [u]
executePlan rs u (a:sigma) =
    nub (concat [ executePlan rs v sigma | v <- image (r_a rs a) u ])

-- Plan is SE or not at a state
stronglyExecutableAt :: Relations -> State -> Plan -> Bool
stronglyExecutableAt _  _ []       = True
stronglyExecutableAt rs u (a:sigma) =
    let next = image (r_a rs a) u
    in  not (null next) &&
        all (\v -> stronglyExecutableAt rs v sigma) next
```

The satisfaction relation $\models$ defines when a formula φ is true in a given state s of an ability map $\mathcal{M}$. Before specifying the semantics, we first introduce two helper functions for the semantics of the Kh operator.

```haskell
-- Given an LST and formula, returns the states that satify said formula
statesSatisifying :: AbilityMap -> Form -> [State]
```

```haskell
statesSatisifying m f = [s | s <- toList (states m), isTrue (m, s) f]

-- Given an LST, find all plans.
findPlans :: AbilityMap -> [Plan]
findPlans m = nub (concatMap (plansFrom depth) (toList (states m)))
  where
    -- For now we limit the depth to avoid infinite loops.
    depth = 5

    -- Flatten transitions
    edges :: [(Action,(State,State))]
    edges = [ (a,(u,v))| (a,rel) <- transitions m, (u,v) <- rel ]

    -- Generate all plans from a given state up to our specified depth
    plansFrom :: Int -> State -> [Plan]
    plansFrom 0 _ = []
    plansFrom d s =
        [ [a] | (a,(x,_)) <- edges, x == s ] -- single action plans
        ++ [ a:p | (a,(x,y)) <- edges, x == s, p <- plansFrom (d-1) y ]

isTrue :: (AbilityMap, State) -> Form -> Bool
isTrue _ T = True
isTrue (m, s) (P p) =
  case lookup s (valuation m) of
    Just props -> p `elem` props
    Nothing -> False
isTrue (m, s) (Neg f) = not (isTrue (m, s) f)
isTrue (m, s) (Conj f g) = isTrue (m, s) f && isTrue (m, s) g
-- KH is NOT local; its truth does not depend on the state at which it is evaluated.
-- KH either holds at all states, or none of them.
isTrue (m, _) (KH f g) =
    any (\a ->
        all (\s ->
            stronglyExecutableAt rs s a
            && all (\t -> isTrue (m,t) g) (executePlan rs s a)
        ) statesF
    ) candidatePlans
  where
    rs = transitions m
    statesF = statesSatisifying m f
    candidatePlans = findPlans m

-- Infix alias for the satisfaction relation
(|=) :: (AbilityMap, State) -> Form -> Bool
(|=) = isTrue
```

Finally, for this section we define the instances of Arbitrary for `Form` and `Arbitrary` respectively. For `Form` we use the `sized` function to ensure that the generated formulas remain finite in size.

```haskell
instance Arbitrary Form where
    arbitrary = sized randomForm where
        -- Helper function to generate random formulas of a given size
        randomForm :: Int -> Gen Form
        randomForm 0 = oneof [P <$> choose (1, 5), return T]
        randomForm n = oneof
            [ P <$> choose (1, 5)
            , return T
            , Neg <$> randomForm (n - 1)
            , Conj <$> randomForm (n `div` 2) <*> randomForm (n `div` 2)
            , KH <$> randomForm (n `div` 2) <*> randomForm (n `div` 2)
            ]

instance Arbitrary AbilityMap where
    arbitrary = do
        n <- choose (1,10)
        let sts = [1..n] -- n states
        rels <- generateRelations n sts
        vals <- mapM generateValuation sts
        return $ LTS (head sts :| tail sts) rels vals
        where
            generateRelations n sts
                | n == 1 = return []
```

```
                | otherwise = do
                    m <- choose (1,n) -- decide how many actions to generate
                    actions <- vectorOf m (choose (1,5))
                    mapM (generateRel sts) actions

             -- for each action a, generate a relation labeled by a
            generateRel sts a = do
                x <- elements sts
                y <- elements (delete x sts) -- avoid loops
                return (a, [(x,y)])

            -- for each state s, generate a list of propositions
            generateValuation s = do
                props <- listOf (choose (1,5))
                return (s, nub props)
```

For now, we haven't had time yet to implement any 'interactive' elements. But it is possible to try out semantics using the definitions by running `stack ghci`. For example:

```
ghci> import Test.QuickCheck
ghci> import Data.List.NonEmpty (NonEmpty(..), toList)

ghci> generate (arbitrary:: Gen AbilityMap)
LTS {
    states = 1 :| [2,3,4,5,6],
    transitions = [(3,[(5,2)]),(4,[(2,4)]),(1,[(5,4)])],
    valuation = [(1,[4,2]),(2,[]),(3,[1]),(4,[3]),(5,[4]),(6,[])]
    }

ghci> m = ...

ghci> isTrue (m, 2::State) (KH (Conj (P 4) (Neg (P 2))) (P 3))
True

ghci> isTrue (m, 2::State) (KH (P 4) (P 1))
False
```

4 Multi Agent

In this section we model the language of Uncertainty-based Knowing How with regularity constraints *reg* L^U_{KH} [DF23]. The paper we reference combines and alters already exisiting languages into the one we model here. The first key difference from the previous language being that it moves from single agent to a multi agent language.

Given a set of proposition letters $Prop$ and a set of agents Agt, where $p \in Prop$ and $i \in Agt$, the language *reg* L^U_{KH} is defined as $\varphi := p \mid \neg\varphi \mid \varphi \wedge \varphi \mid Kh_i(\psi, \varphi) \mid T$ s.t. $Kh_i(\psi, \varphi)$ is the modality expressing "when ψ is the case, the agent a knows how to make φ true".

```
module MultiAgent where

import SingleAgent

type Agent = Int

data RegForm = P Proposition | Neg RegForm | Conj RegForm RegForm | KH Agent RegForm
    RegForm | T
```

```
    deriving (Eq, Show, Ord)
```

Now we define the automaton used in the model-checking procedure. This is a non-deterministic automaton of the form

$$\mathcal{A} = (Q, Act, \delta, I, F),$$

where, in our setting:

- $Q = S$, i.e. the automaton states coincide with the states of the LTS;
- Act is the set of actions;
- δ is represented as a transition relation, or equivalently as a mapping from a state-action pair to a set of successor states;
- I is the singleton containing the initial state;
- F is the singleton containing the final state.

Note that since the automaton here refers to a non-deterministic automaton, the transition is modeled as a relation and not a function. Moreover, $\forall t, t' \in Q$ and $a \in Act$ we have that $t \xrightarrow{a} t' \in \delta \Leftrightarrow (t, t') \in R_a$ (as previously defined for single agent).

Since both I and F are singletons we define both as a `State`, rather than a `[State]`, to avoid unintended behaviour.

```
type Successors = [State]
type ActionAtState = (State, Action)

data Automaton = ATMN {
    states :: [State],
    actions :: [Action],
    transitions :: [(ActionAtState, Successors)],
    initial :: State,
    final :: State
} deriving (Eq, Show, Ord)
```

5 Tests for Semantics

We now use the library QuickCheck to randomly generate models and states to verify the fundamental logical facts within the boolean fragment of our implementation.

```
module Main where

import Test.QuickCheck
import Data.List.NonEmpty (toList)
import SingleAgent
import Test.Hspec
```

The following uses the HSpec library to define our test suite. These tests ensure that our model checker respects classical propositional identities. To perform these tests, we first define an auxiliary type `PointedModel` and an `Arbitrary` instance to pick a random state from a generated model.

```
newtype PointedModel = PM (AbilityMap, State) deriving (Show)

instance Arbitrary PointedModel where
    arbitrary = do
        m <- arbitrary
        s <- elements (toList (states m))
        return (PM (m, s))
```

We verify the following principles which should hold for any boolean formula φ:

- **Double Negation Elimination**
  - $\neg\neg\varphi \equiv \varphi$
- **Commutativity of Conjunction**
  - $\varphi \wedge \psi \equiv \psi \wedge \varphi$
- **Identity of Conjunction**
  - $\varphi \wedge \top \equiv \varphi$

```
-- Since Kh is currently undefined, we use a helper function to filter out formulas
    containing the Kh modality.
isBoolean :: Form -> Bool
isBoolean (P _)        = True
isBoolean T            = True
isBoolean (Neg f)      = isBoolean f
isBoolean (Conj f1 f2) = isBoolean f1 && isBoolean f2
isBoolean (KH _ _)     = False

-- Check if two formulas are equivalent in a specific model and state
isEquivalent :: (AbilityMap, State) -> Form -> Form -> Bool
isEquivalent (m, s) f1 f2 = isTrue (m, s) f1 == isTrue (m, s) f2

main :: IO ()
main = hspec $ do
  describe "Boolean Semantics" $ do
    it "Double Negation Elimination: Neg (Neg f) should be equivalent to f" $
      property $ \(PM (m, s)) f ->
        isBoolean f ==> isEquivalent (m, s) (Neg (Neg f)) f

    it "Commutativity of Conjunction: f1 ^ f2 should be equivalent to f2 ^ f1" $
      property $ \(PM (m, s)) f1 f2 ->
        (isBoolean f1 && isBoolean f2) ==> isEquivalent (m, s) (Conj f1 f2) (Conj f2 f1)

    it "Identity of Conjunction: f ^ T should be equivalent to f" $
      property $ \(PM (m, s)) f ->
        isBoolean f ==> isTrue (m, s) (Conj f T) === isTrue (m, s) f
```

6 Conclusion

Finally, we can see that [?] is a nice paper.

References

[AFSVQ21] C. Areces, R. Fervari, A. R. Saravia, and F. R. Velázquez-Quesada. Uncertainty-based semantics for multi-agent knowing how logics. In *Proceedings of the 18th*

Conference on Theoretical Aspects of Rationality and Knowledge (TARK 2021), volume 335 of *EPTCS*, pages 23–37, 2021.

[DF23] S. Demri and R. Fervari. Model-checking for ability-based logics with constrained plans. In *Proceedings of the 37th AAAI Conference on Artificial Intelligence (AAAI-23)*, volume 37, pages 6305–6312, 2023.

[Wan15] Y. Wang. A logic of knowing how. In *Logical Reasoning and Interaction (LORI'15)*, volume 9394 of *Lecture Notes in Computer Science (LNCS)*, pages 392–405. Springer, 2015.